# QRAFT

## Quantum-Resistant
## Anonymous Frame Transactions

Use one unified private balance to send funds anonymously.

Built with ERC-8286 PQ wallets and EIP-8141 transactions.

# Tornado Cash Classic: limitations

| | |
|---|---|
| Withdrawals depend on a centralised relayer | In the relayer route, a third-party service submits your transaction. |
| Paying another wallet requires a withdrawal | Funds leave the private pool and arrive at a public address. |
| Direct RPC requests expose your IP | Onchain privacy does not hide your network identity. |
| Funds are split into fixed-value notes | Each deposit has its own note; exact payments are awkward. |
| No built-in post-quantum wallet | Classic does not provide PQ account signatures or receiving encryption. |

# What QRAFT adds

| | |
|---|---|
| One unified private balance | Automatic note selection, exact amounts and private change. |
| No application relayer | EIP-8141 goes directly through Anon RPC to the alternative mempool. Paymaster sponsors setup and gas. |
| Pay privately without withdrawing | Encrypted payments move between users inside the pool. |
| IP protection through Nym | Anon RPC routes reads and submissions through the mixnet. |
| ERC-8286 PQ wallets | ML-DSA account signatures and ML-KEM receiving encryption. |

# Demo

# Post-quantum keypairs in use

Two PQ keypairs and one spending secret derive from one random master secret.

### 1 · Signing keypair · ML-DSA-44

**Private key:** signs account setup and public deposits.
**Public key:** lets the onchain verifier check authorization.

### 2 · Receiving keypair · ML-KEM-768

**Public key:** shared in your receiving address.
**Private key:** enables your wallet to decrypt incoming payment notes.

### 3 · Spending secret · not a keypair

Lets you spend your private notes. Your device uses it to prove ownership and create a nullifier—a unique marker that prevents spending the same note twice. The secret stays on your device.

### 4 · Passkey · device credential

A separate WebAuthn credential unlocks the wallet. Its PRF output derives an AES key that protects the master secret and encrypted backup.

---

**Receiving address:** format version + chain ID + genesis block hash + private-pool contract address + owner commitment + public ML-KEM receiving key + checksum.

# Add, send and receive private funds

A note is a private record of funds you can spend. The wallet manages these records for you.

### 1 · Add ETH to your private balance

Deposit ETH from your public account into the pool. Your wallet creates an encrypted note for that amount. It adds this note to your existing funds and shows one private balance.

### 2 · Choose who to pay and how much

Paste the receiver’s QRAFT receiving address and enter an amount. Your wallet automatically selects enough notes and proves that you own the funds.

### 3 · The receiver gets a new note

The payment note is encrypted using the receiver’s public receiving key and published by the pool. Their wallet uses its private receiving key to read it and adds the amount to their private balance.

### 4 · The change stays yours

The used notes are marked spent. Any leftover value becomes a new encrypted note for you.
**Use 0.05 ETH in notes to send 0.03 ETH → receive 0.02 ETH back as private change.**

ETH stays in the shared pool during a private payment. The encrypted notes record who can spend it. ETH leaves the pool only when someone withdraws.

# From deposit to private payment

**Account setup:** browser signs with ML-DSA → factory verifies and deploys your ERC-8286 PQ account → paymaster funds the 0.012 ETH stake and gas.

01 · FUNDS TO PRIVATE BALANCE

### Public account
ML-DSA authorizes the amount of ETH deposited.

→

### Anon RPC → node
Nym carries EIP-8141 frames to the alternative mempool.

→

### Verify → sponsor → execute
Account verifies signature; paymaster covers gas; pool verifies deposit proof.

→

### Shared pool
Holds ETH. Records a commitment and encrypted deposit note.

02 · PRIVATE SEND — SHARED PROOF ACCOUNT IS THE TRANSACTION SENDER

### Sender’s device
Selects notes to cover the payment, encrypts payment and change, and generates a local proof.

→

### Anon RPC → node
Browser submits the proof-authorized EIP-8141 transaction. No application relayer.

→

### Three native frames
1. Shared account validates proof. 2. Paymaster approves gas. 3. Pool executes payment.

→

### Pool settlement
Consumes nullifiers; records payment and change commitments plus encrypted notes. ETH stays in pool.

**Receive:** scan pool events → decrypt with ML-KEM + AES → verify notes → filter spent nullifiers → update unified balance.
**Withdraw:** prove a private spend → pool releases ETH to the chosen public address; remaining change stays private.